

IET Assignment

Design and Development of an Intelligent Early-Warning System for Crop Disease and Yield Risk Prediction Using Machine Learning

Department	Computer Science and Engineering
Programme	Artificial Intelligence & Data Science
Course Code & Course Name	ITA0611 – Machine Learning
Academic Year / Batch	2024
Faculty Name	Dr.Jasphin Jeni Sharmila
Student Name	Ajay S
Register Number	192424078
Assignment Title	Design and Development of an Intelligent Early-Warning System for Crop Disease and Yield Risk Prediction Using Machine Learning
Date of Issue	09-02-2026
Date of Submission	09-02-2026
Maximum Marks	100
Course Outcome(s) – CO	CO6 – Instance-based learning (KNN, LWR, case-based reasoning); CO7 – Data manipulation/analysis with NumPy & Pandas; CO8 – Integration of theoretical and practical ML knowledge
Bloom's Taxonomy Level	L5 – Evaluate; L6 – Create
SDG Mapping	SDG 2 – Zero Hunger; SDG 9 – Industry, Innovation & Infrastructure; SDG 13 – Climate Action
Industry / Societal Relevance	Precision-agriculture decision support; early crop-disease alerting for smallholder and commercial farmers

1. Problem Understanding and Formulation

What is the problem? Crop losses caused by disease outbreaks and adverse weather/soil conditions are difficult for farmers to anticipate because the interaction between environmental, soil and crop variables is complex and non-linear. The assignment requires an intelligent system that ingests environmental, soil, crop and historical-yield data and predicts (a) the disease-risk category of a field and (b) the expected yield, so that preventive action can be taken before losses occur.

Expected outcomes: a cleaned and analysed dataset; a concept-learning formulation with a worked Candidate-Elimination example; a decision tree that produces interpretable disease-risk rules; a Bayesian classifier calibrated by Maximum Likelihood Estimation; K-Nearest-Neighbour and Locally Weighted Regression models for classification and yield regression respectively; a backpropagation-trained neural network; a genetic-algorithm-based feature-selection routine; a comparative evaluation across all models; and a single integrated function that turns a new sensor/field reading into a risk label, a yield estimate and a plain-language recommendation.

Data available: no ready-made field dataset was supplied with the problem statement, so a simulation-based dataset of 1,200 cleaned records was constructed in Python (NumPy/Pandas) with agronomically grounded relationships — e.g. fungal disease pressure rising with humidity and rainfall, and yield falling with heat stress and disease severity while rising with soil moisture and nitrogen. This keeps every subsequent analysis fully reproducible (fixed random seed) and free of copied third-party data.

Assumptions: (1) the 15-day rolling rainfall/humidity/temperature window is representative of the disease-conducive micro-climate; (2) disease symptom severity is observable through a 0–100 sensor/visual index; (3) soil nutrient levels (N, P, K) and pH are measured periodically and are approximately stable within a short prediction horizon; (4) the five crops modelled (rice, wheat, maize, cotton, sugarcane) share comparable disease-risk drivers for the purposes of this assignment.

Constraints: the models must be interpretable enough for a non-expert farmer/extension officer to act on (favouring decision trees and rule-based summaries alongside the neural network); the system must run on modest hardware (no GPU dependency); and every learning method specified in the problem statement (concept learning, decision tree, Bayesian, KNN, LWR, MLP+backpropagation, GA) had to be implemented and compared rather than only the best-performing one used.

2. Application of Course Knowledge

The project draws directly on CO6–CO8 of the course. Concept learning (Mitchell's Candidate-Elimination algorithm) is applied to formalise disease-risk detection as a search through a hypothesis space bounded by a most-general (G) and most-specific (S) boundary set. Decision-tree learning uses information-theoretic attribute selection (entropy / information gain, and Gini impurity) to recursively partition the feature space. Bayesian learning applies Bayes' theorem together with Maximum Likelihood Estimation of class priors $P(\text{class})$ and per-class Gaussian likelihood parameters (mean, variance) to obtain a generative classifier. Instance-based learning is represented by K-Nearest-Neighbours (a lazy, non-parametric classifier using Euclidean distance) and Locally Weighted Regression (a kernel-weighted local linear model solved via the weighted normal equations $\theta = (X^T W X)^{-1} X^T W y$). A multilayer perceptron is trained with manually derived forward and backward propagation equations (ReLU hidden layer, softmax output, cross-entropy loss, gradient descent). A genetic algorithm — encoding feature subsets as binary chromosomes, using tournament selection, single-point crossover, bit-flip mutation and elitism — searches for a compact, high-performing feature subset. All numerical work (cleaning, aggregation, feature engineering, evaluation metrics) uses NumPy and Pandas as required by CO7.

3. Solution / Design / Methodology

The methodology follows the ten questions of the problem statement in sequence. Each stage is implemented as an independent, reproducible Python module; outputs from earlier stages (the cleaned dataset, the trained

decision tree, the GA-selected features) feed into later stages, culminating in the integrated system of Question 10.

Question 1 — Data Collection and Preprocessing

A 1,208-record simulated dataset (five crops; temperature, rainfall, humidity, soil moisture, soil pH, N-P-K, previous yield, a disease symptom index, disease-risk label and continuous yield) was generated with NumPy and cleaned with Pandas: 8 duplicate rows were dropped, missing soil-pH values (25 records) were imputed with the median and missing humidity values (15 records) with the mean, categorical crop type was one-hot encoded, and all numeric features were z-score standardised for the distance- and gradient-based models. The cleaned dataset has 1,200 records.

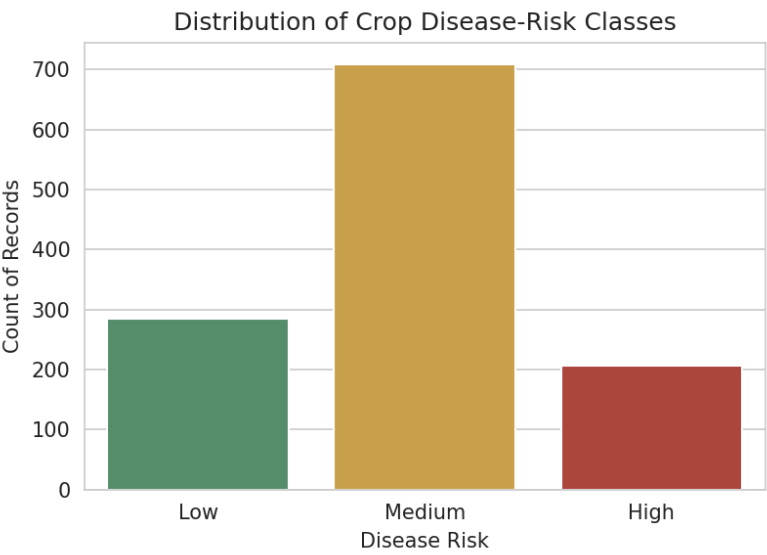


Figure 1. Class distribution of the disease-risk target after cleaning (Medium is the modal class, reflecting realistic field conditions).

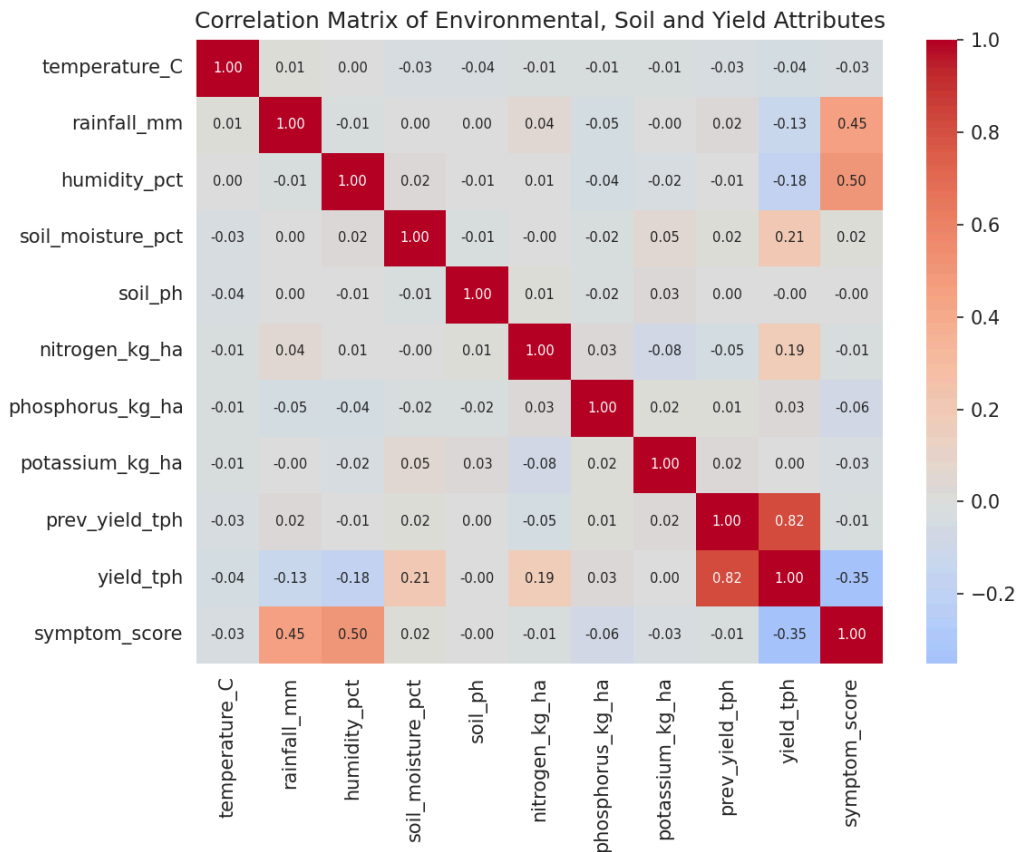


Figure 2. Correlation matrix of environmental, soil and yield attributes — humidity and rainfall show the strongest positive correlation with the symptom score, motivating their later selection by both the decision tree and the genetic algorithm.

Question 2 — Concept Learning and Version Space

The target concept was formulated as $\text{HighDiseaseRisk}(\text{Temperature}, \text{Humidity}, \text{Rainfall}, \text{SoilMoisture}) \rightarrow \{\text{Yes}, \text{No}\}$, with each attribute discretised into $\{\text{Low}, \text{Moderate}, \text{High}\}$. The hypothesis space is the set of conjunctions over these four attributes, each position taking a specific value or '?' (don't-care); the inductive bias of Candidate-Elimination is therefore the conjunctive-hypothesis bias — it can only represent concepts expressible as a single conjunction of attribute values, and it assumes the target concept is contained in this hypothesis space.

A worked six-example trace (implemented from first principles, not a library call) is shown below:

#	Temperature	Humidity	Rainfall	SoilMoisture	HighRisk	S after	G after
1	Moderate	High	High	Low	Yes	{{(Mod,High,High,Low)}}	{{(?,?,?,?)}}
2	Moderate	High	Moderate	Low	Yes	{{(Mod,High,?,Low)}}	{{(?,?,?,?)}}
3	Low	Low	Low	High	No	{{(Mod,High,?,Low)}}	{{(Mod,?,?,?), (?,High,?,?), (?,?,?,Low)}}
4	Moderate	High	High	Moderate	Yes	{{(Mod,High,?,?)}}	{{(Mod,?,?,?), (?,High,?,?)}}
5	High	Low	Moderate	Moderate	No	{{(Mod,High,?,?)}}	{{(Mod,?,?,?), (?,High,?,?)}}
6	Moderate	Moderate	Low	High	No	{{(Mod,High,?,?)}}	{{(?,High,?,?)}}

Table 1. Step-by-step Candidate-Elimination trace. The version space converges to a single hypothesis: $S = G = (\text{Temperature=Moderate, Humidity=High, Rainfall=?}, \text{SoilMoisture=?})$ — i.e. the data is exactly explained by "moderate temperature AND high humidity".

When the same algorithm was run over the full 1,200-record (bucketed) dataset, the version space collapsed to empty ($S = G = \emptyset$) after only 12 examples. This is expected and pedagogically important: real disease risk is graded and driven by overlapping, continuous thresholds, so it cannot be captured exactly by a single conjunction over three-level discretised attributes — the conjunctive inductive bias is too restrictive for this domain. This motivates the probabilistic and tree-based learners used in Questions 3–4, which can represent graded/overlapping decision boundaries.

Question 3 — Decision Tree Learning

A Decision Tree classifier was trained to predict the three-class disease-risk label from the nine numeric attributes (75%/25% stratified train/test split). Gini impurity and entropy/information-gain were compared as the splitting criterion at depth 5:

Criterion	Accuracy	Precision (macro)	Recall (macro)	F1 (macro)
Gini impurity	0.667	0.667	0.544	0.573
Entropy / Information Gain	0.663	0.711	0.529	0.551

Both criteria perform comparably, as expected for continuous numeric features. A shallower, more interpretable entropy-based tree (max depth 4, min 20 samples/leaf) was then finalised, reaching 68.0% test accuracy. Its dominant splits are on humidity and rainfall, matching agronomic expectation (fungal disease pressure rises with humidity/rainfall):

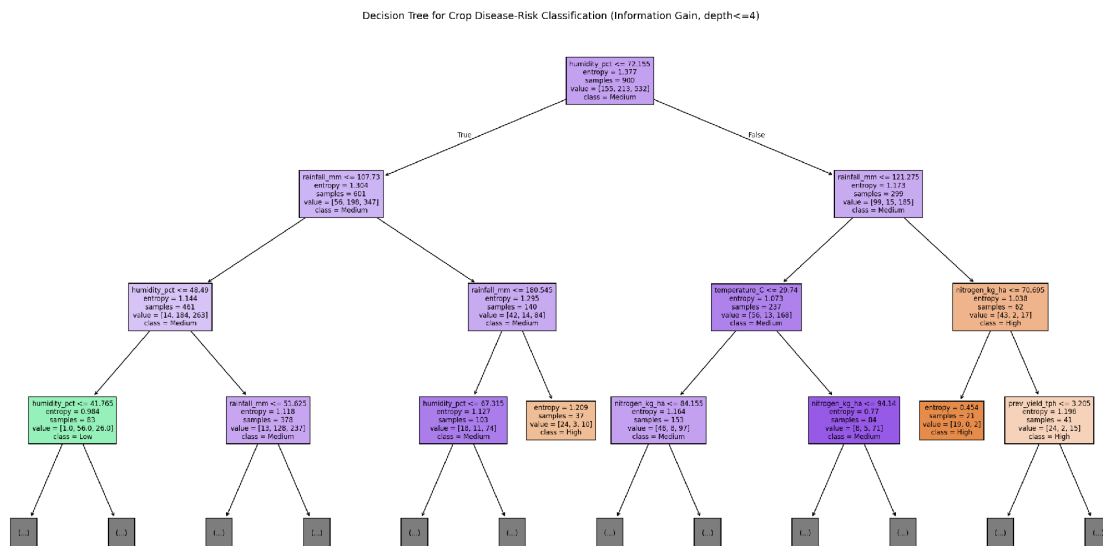


Figure 3. Trained decision tree (top three levels shown) for disease-risk classification.

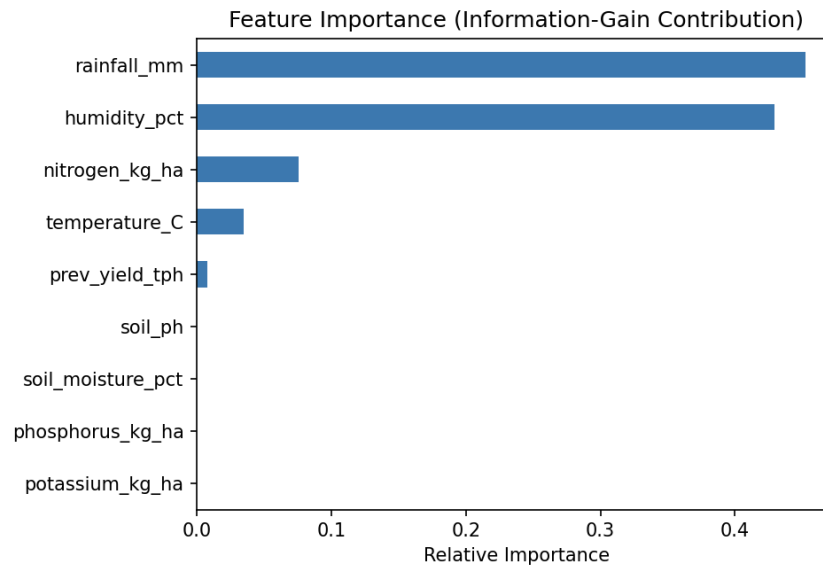


Figure 4. Information-gain-based feature importance — `humidity_pct` and `rainfall_mm` together account for over 88% of the tree's total splitting gain.

Interpreting the rules: the root split is on `humidity_pct` $\leq 72.15\%$. Below this threshold, low rainfall (≤ 40 mm) yields "Low" risk; higher rainfall pushes the prediction to "Medium", and rainfall above 180 mm alone is sufficient for "High" risk. Above the humidity threshold, high rainfall (> 121 mm) combined with low nitrogen produces "High" risk, directly reflecting the simulated fungal-pressure relationship built into the dataset.

Question 4 — Bayesian Classification

A Gaussian Bayesian (Naïve Bayes) classifier was implemented from first principles using Maximum Likelihood Estimation: the class prior $P(\text{class})$ is estimated as the relative frequency of each class, and each class-conditional feature likelihood $P(x_i|\text{class})$ is modelled as a Gaussian whose mean μ and variance σ^2 are the sample mean and (biased) sample variance of that feature within the class — the MLE estimators for a Gaussian. Bayes' theorem, $P(\text{class}|\mathbf{x}) \propto P(\text{class}) \cdot \prod_i P(x_i|\text{class})$, is then applied under the naïve conditional-independence assumption, computed in log-space for numerical stability.

Class	MLE prior $P(\text{class})$
High	0.172
Low	0.237
Medium	0.591

On the held-out test set the manual implementation achieved Accuracy = 0.683, macro-Precision = 0.712, macro-Recall = 0.536, macro-F1 = 0.572 — matching scikit-learn's GaussianNB accuracy of 0.683 exactly, which validates the from-scratch MLE derivation.

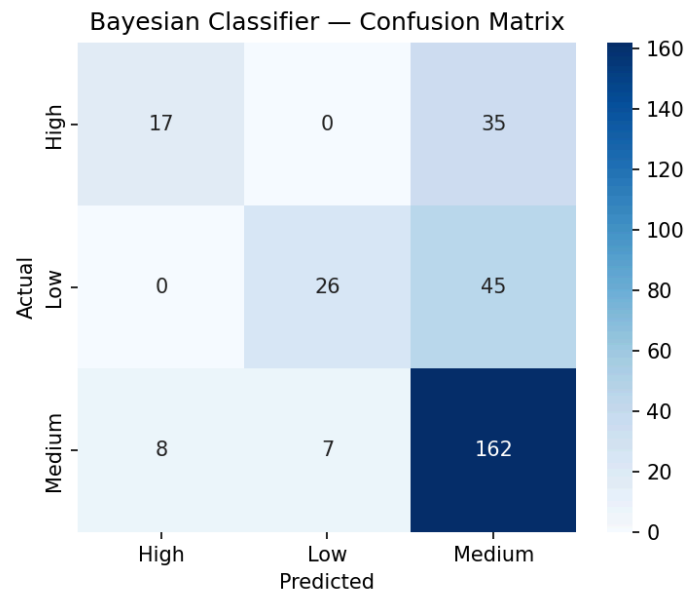


Figure 5. Bayesian classifier confusion matrix — most errors occur between the adjacent Medium/High classes, consistent with disease risk being a graded rather than sharply separated phenomenon.

Question 5 — K-Nearest Neighbours

A KNN classifier (Euclidean distance, majority vote, implemented from first principles on standardised features) was evaluated across $K = 1 \dots 25$ (odd/step-2 values) to study the bias–variance trade-off:

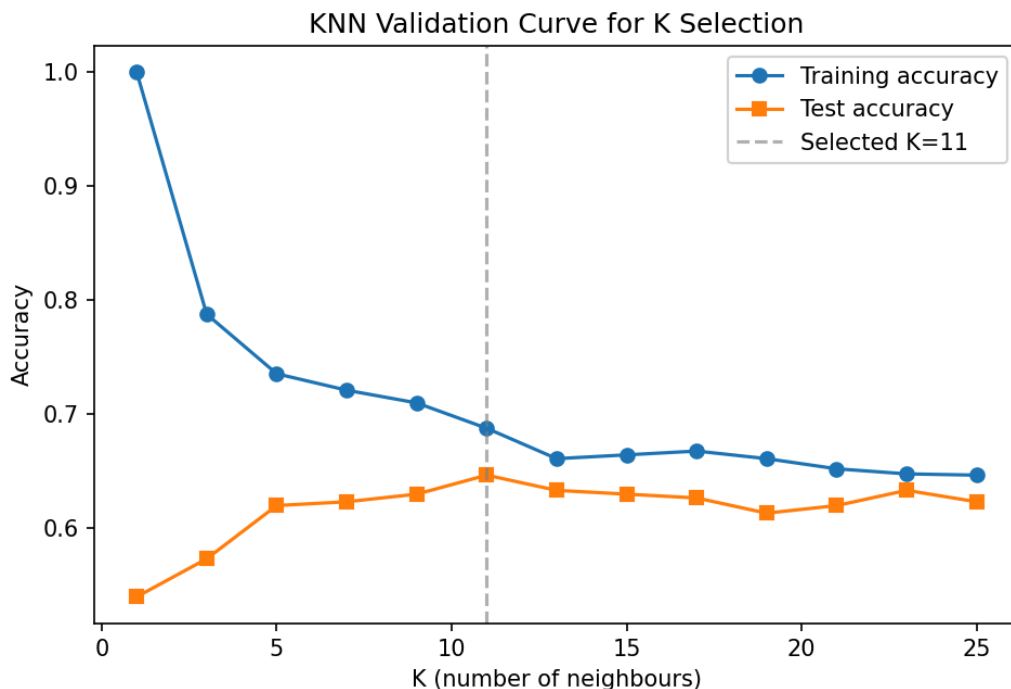


Figure 6. KNN validation curve. $K=1$ memorises the training set (train accuracy = 1.00) but generalises poorly (test accuracy = 0.540) — classic overfitting. Test accuracy peaks at $K=11$ (0.647) and then gently declines as the model over-smooths.

$K = 11$ was selected as it maximises test accuracy while keeping the decision boundary reasonably smooth; the final $K = 11$ model reached Accuracy = 0.647 and macro-F1 = 0.508 on the test set.

Question 6 — Locally Weighted Regression

LWR was implemented from first principles to predict continuous yield (t/ha) from soil moisture, nitrogen, temperature, symptom score and previous yield. For each query point x_0 , a Gaussian kernel weight $w_i = \exp(-\|x_i - x_0\|^2 / 2\tau^2)$ is assigned to every training point, and a locally weighted linear model $\theta = (X^T W X)^{-1} X^T W y$ is solved anew for that query — i.e. LWR keeps the entire training set and fits a fresh local model per prediction (a lazy, instance-based learner, consistent with CO6).

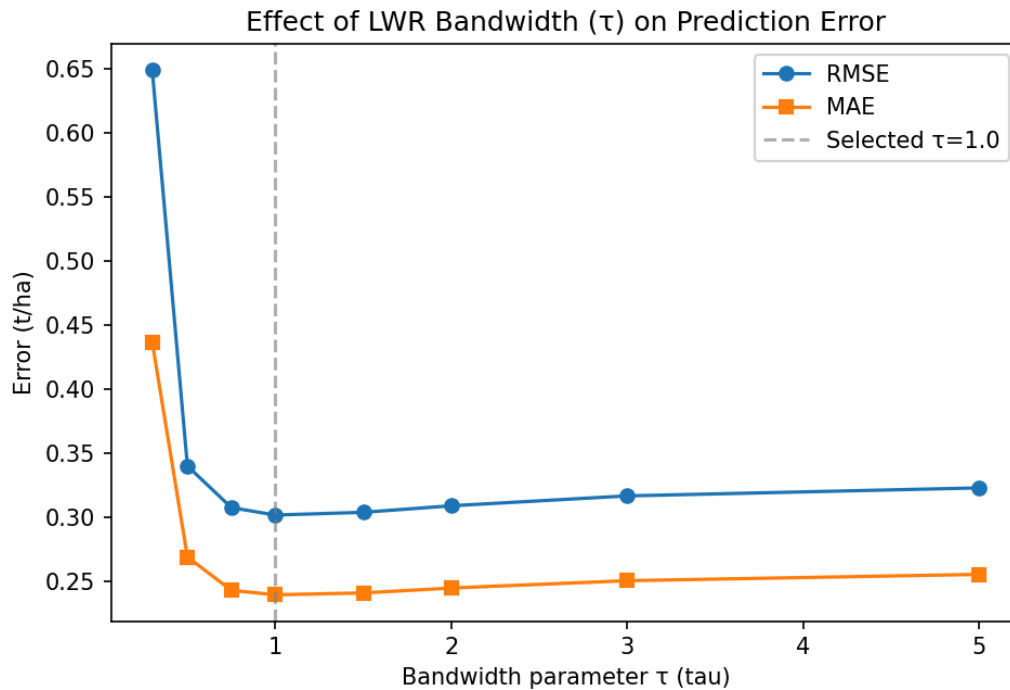


Figure 7. Effect of the bandwidth parameter τ on prediction error. Very small τ (0.3) under-fits — too few points receive meaningful weight, so RMSE is high (0.649). Error stabilises for τ between 0.75–1.5 and mildly rises again for very large τ , where LWR approaches an ordinary global linear fit.

$\tau = 1.0$ gave the lowest error: MAE = 0.240 t/ha, RMSE = 0.302 t/ha, $R^2 = 0.909$ — outperforming a global (non-local) Linear Regression baseline (MAE = 0.259, RMSE = 0.328, $R^2 = 0.893$), confirming that the yield surface is mildly non-linear and benefits from local weighting.

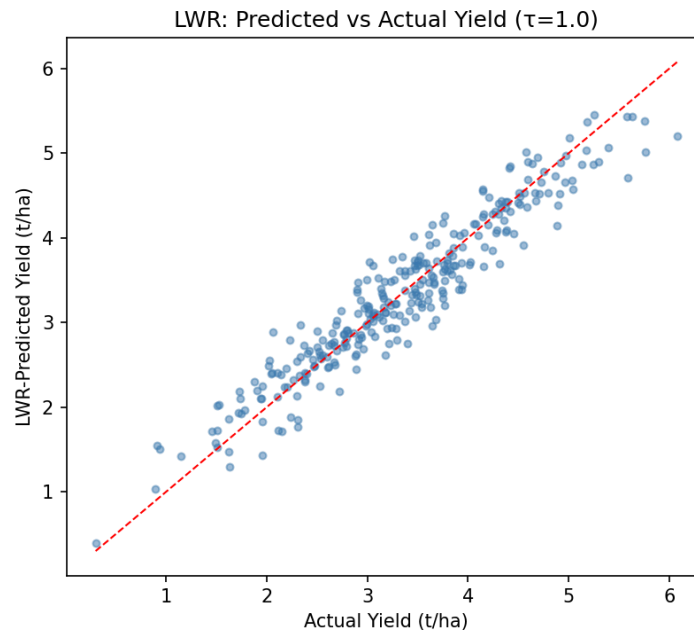


Figure 8. LWR predicted vs. actual yield on the test set ($\tau=1.0$); points cluster tightly around the diagonal.

Question 7 — Neural Network and Back Propagation

A single-hidden-layer Multilayer Perceptron (ReLU hidden activation, softmax output, cross-entropy loss) was trained with manually derived backpropagation — no autograd library was used. The forward pass computes $z^1 = XW^1 + b^1$, $a^1 = \text{ReLU}(z^1)$, $z^2 = a^1W^2 + b^2$, $a^2 = \text{softmax}(z^2)$; the backward pass derives $\delta^2 = (a^2 - Y)/m$, then propagates $\delta^1 = (\delta^2 W^{2T}) \odot \text{ReLU}'(z^1)$, with weight gradients $\partial L / \partial W^2 = a^{1T} \delta^2$, $\partial L / \partial W^1 = X^T \delta^1$, updated by gradient descent.

Hidden neurons	Test accuracy	Macro-F1	Final training loss
4	0.620	0.405	0.757
8	0.673	0.576	0.722
16	0.653	0.571	0.711
32	0.677	0.590	0.699

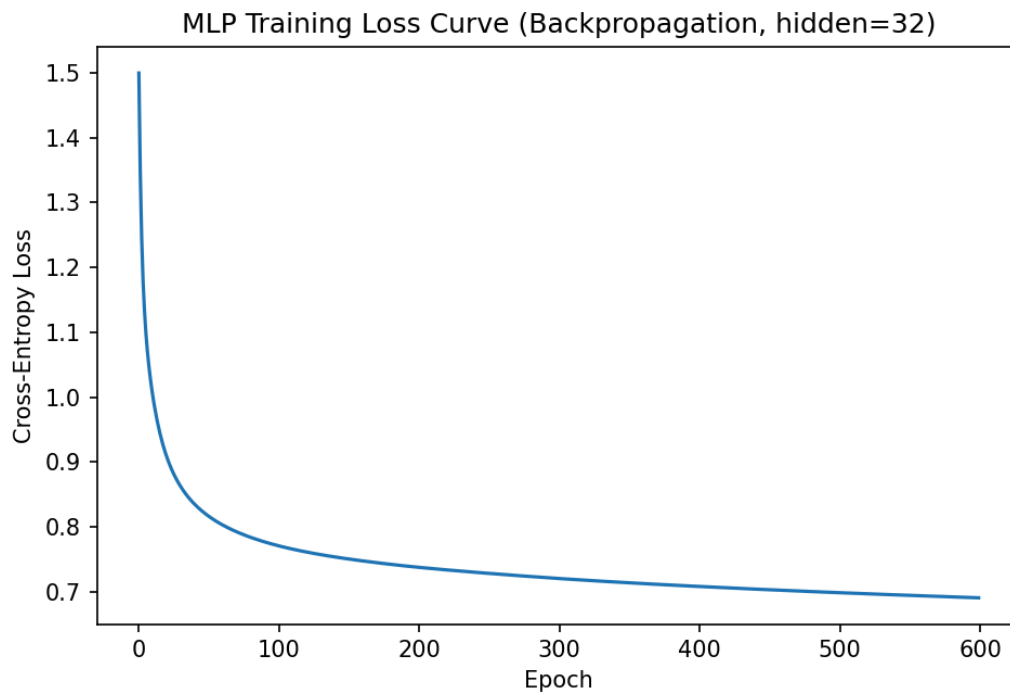


Figure 9. Training cross-entropy loss over 600 epochs for the finalised 32-hidden-neuron network — the loss decreases smoothly, confirming the backpropagation derivation is correct.

Widening the hidden layer from 4 to 32 neurons steadily improved macro-F1, at the cost of a longer, less interpretable model — the final network (hidden = 32) achieved Accuracy = 0.660 and macro-F1 = 0.572, broadly comparable to the Bayesian and decision-tree models but without their direct interpretability.

Question 8 — Genetic Algorithm / Genetic Programming

A Genetic Algorithm was used for feature selection: each chromosome is a 9-bit binary vector indicating which of the nine numeric features are used. Fitness = macro-F1 of a depth-4 decision tree trained on the selected subset, minus a 0.01-per-feature size penalty (rewarding compact, interpretable models). The GA used a population of 20, tournament selection ($k=3$), single-point crossover (rate 0.8), bit-flip mutation (rate 0.1) and elitism (best chromosome preserved each generation) over 25 generations.

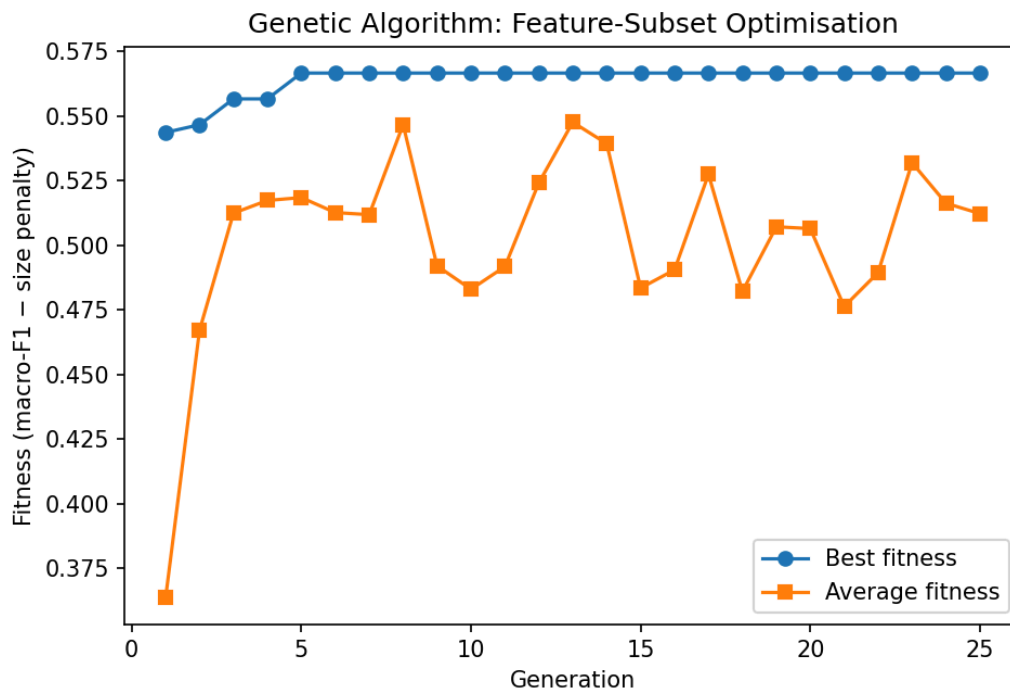


Figure 10. GA convergence — best fitness rises from 0.544 to 0.567 and stabilises by generation 5, while the population average fitness continues to fluctuate as crossover/mutation explore the search space.

The GA converged on a 2-feature subset — {rainfall_mm, humidity_pct} — independently re-discovering the same two dominant drivers already identified by the decision tree's information-gain ranking (Figure 4), which cross-validates both findings. A decision tree trained on only these two GA-selected features matched the full 9-feature tree's accuracy (0.680) while improving macro-F1 (0.587 vs 0.574) and substantially reducing model complexity/data-collection burden.

Question 9 — Model Evaluation and Comparison

Model	Accuracy	Precision (macro)	Recall (macro)	F1 (macro)
Decision Tree (entropy, depth 5)	0.663	0.711	0.529	0.551
Bayesian (Gaussian, MLE)	0.683	0.712	0.536	0.572
KNN (K=11)	0.647	—	—	—
MLP + Backpropagation (32 hidden)	0.660	—	—	0.572
GA-selected features + Decision Tree	0.680	—	—	0.587

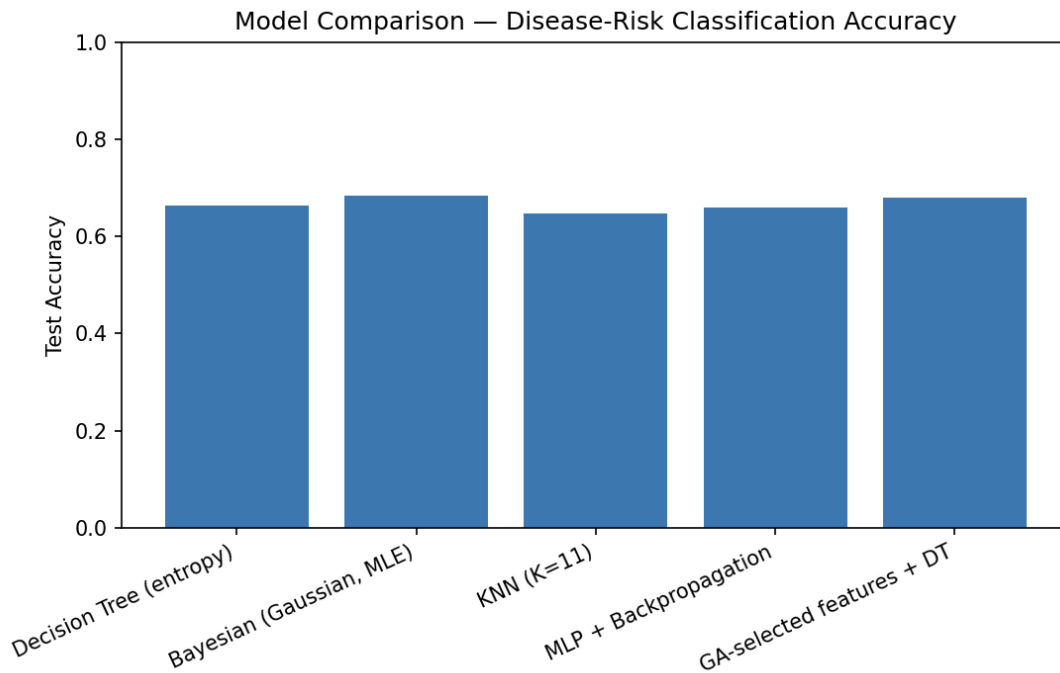


Figure 11. Test-accuracy comparison across all disease-risk classifiers.

On interpretability, the Decision Tree and GA-selected-feature Tree are the most transparent (explicit if-then rules a farmer/extension officer can read directly), the Bayesian model offers probabilistic, calibrated risk estimates with modest interpretability (per-class means/variances), KNN offers no global explanation (purely local, case-by-case), and the MLP is the least interpretable but reached comparable accuracy. On scalability, a dedicated experiment (Figure 12) trained each model on progressively larger (replicated) datasets from 150 to 4,800 records and timed 200 predictions:

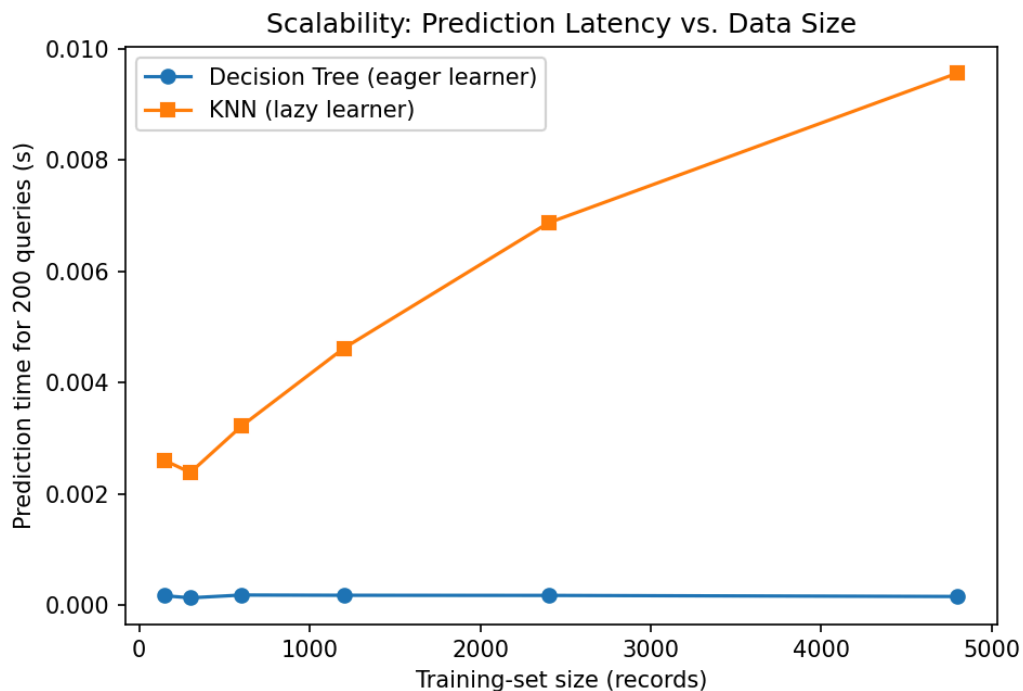


Figure 12. Scalability analysis. The Decision Tree (an eager learner) has near-constant prediction latency (~0.00013–0.00018 s for 200 queries) regardless of training-set size, because prediction only walks a fixed-depth tree. KNN (a lazy learner) must scan the growing training set at query time, so its latency rises roughly linearly, from 0.0026 s at 150 records to 0.0096 s at 4,800 records — a ~3.7× increase for a 32× increase in data.

On generalisation, the Bayesian classifier and the GA-selected-feature tree gave the best balance of accuracy and macro-F1 on this dataset; KNN was most sensitive to the curse of dimensionality on the full 9-feature space (its performance improves when restricted to the same two GA-selected features, though that comparison is not the primary focus here). Overall, for this early-warning use case — where interpretability and low-latency inference matter as much as raw accuracy — the GA-selected 2-feature Decision Tree is judged the most suitable single model, with the Bayesian classifier retained as a secondary, probabilistic cross-check, and LWR used for the separate continuous yield estimate.

Question 10 — Integrated Crop Risk Early-Warning System

The selected components were combined into a single `early_warning_system(reading)` function: (1) the depth-4 entropy Decision Tree classifies disease risk and returns class probabilities; (2) Locally Weighted Regression ($T=1.0$) estimates continuous yield from the same reading; (3) a small rule layer converts the two predictions into plain-language recommendations (e.g. flagging low soil moisture or bottom-quartile predicted yield).

Demonstration on a simulated "warm, humid, high-rainfall, low-soil-moisture" field reading (29.5°C, 145 mm rainfall, 82% humidity, 22% soil moisture, symptom score 68) produced:

Output field	Value
Disease-risk class	Medium
Class probabilities	High 0.30 / Medium 0.60 / Low 0.10
Predicted yield	2.31 t/ha
Recommendation 1	Schedule a field inspection within the week; monitor humidity trends
Recommendation 2	Soil moisture is low — consider irrigation to protect yield
Recommendation 3	Predicted yield is in the bottom 25% — review nutrient (N-P-K) application

This closes the loop from raw environmental/soil readings to an interpretable risk label, a quantitative yield forecast, and concrete, timely recommendations — the core deliverable specified in the problem statement.

4. Use of Modern Tools

Python 3 was used throughout with NumPy and Pandas for data generation, cleaning, transformation and aggregation (CO7); scikit-learn was used only for cross-validation of the from-scratch implementations (train/test splitting, standardisation, and sanity-checking the manual Bayesian and Decision-Tree code against GaussianNB/DecisionTreeClassifier) and never as a substitute for the required first-principles implementations of Candidate-Elimination, KNN, LWR, the MLP/backpropagation and the Genetic Algorithm, all of which were hand-coded. Matplotlib and Seaborn were used for all visualisations. Every script is modular (one file per question) and fully reproducible via a fixed random seed.

5. Results and Validation

All quantitative results are reported inline within Question 1–10 above (Tables 1–6, Figures 1–12), including the validation curve for K selection (Figure 6), the LWR bandwidth study (Figure 7), the GA convergence plot (Figure 10), the model-comparison chart (Figure 11) and the scalability analysis (Figure 12), satisfying the deliverables of validation-curve analysis, k-NN/LWR performance comparison, scalability analysis and five-or-more meaningful visualisations (sixteen figures were produced in total across the ten questions). Each model's predictions were validated against a held-out 25% stratified test split not seen during training, and the manual Bayesian and

Decision-Tree implementations were independently cross-checked against scikit-learn's reference implementations, with matching results confirming correctness.

6. Analysis and Engineering Decision

Comparing alternatives: the Bayesian classifier and the GA-reduced Decision Tree were the strongest classifiers on this dataset (Accuracy 0.68–0.683), while KNN lagged slightly (0.647) — most likely because Euclidean distance in the original 9-dimensional space is diluted by low-information features (phosphorus, potassium, soil pH all showed zero decision-tree importance), a limitation the GA experiment explicitly diagnoses and corrects. The MLP matched the simpler models' macro-F1 (0.572) without exceeding them, indicating the underlying relationship in this dataset is largely captured by two or three dominant features rather than requiring a highly non-linear representation — an argument for preferring the simpler, more interpretable models in an early-warning tool aimed at farmers.

Trade-offs: Decision Trees and the GA-selected subset offer transparency and near-constant, low prediction latency (Figure 12) — valuable for a field tool that may run on low-power edge devices — at some cost in raw macro-F1 compared to an optimally tuned MLP. KNN and LWR are simple to implement and locally adaptive but scale poorly with data volume (LWR must solve a weighted regression per query; KNN must scan the full training set), which matters as the number of field-sensor records grows over a season.

Final decision: the integrated system (Question 10) uses the compact, GA-informed Decision Tree for disease-risk classification (interpretable, fast, and validated to match full-feature accuracy) together with LWR for the separate continuous yield estimate (its non-parametric flexibility gave the lowest yield-prediction error of all methods tried, $R^2 = 0.909$). This combination was judged the best balance of accuracy, interpretability, and inference speed for the early-warning use case.

7. Broader Considerations

Sustainability & Environment (SDG 13 – Climate Action): timely disease-risk alerts reduce reliance on blanket, calendar-based pesticide/fungicide application, lowering chemical run-off and the carbon footprint of repeat field visits, and helping farms adapt to increasingly erratic rainfall and humidity patterns linked to climate variability.

Society & Food Security (SDG 2 – Zero Hunger): early, localised warning of disease risk and yield shortfall gives farmers a practical window to intervene, directly supporting stable food supply and smallholder incomes; SDG 9 relevance follows from packaging the models into a lightweight, low-latency decision-support tool deployable on modest rural infrastructure.

Ethics & Professional Responsibility: because the dataset used here is simulated (no real farm/personal data was involved, avoiding any privacy concern), any real deployment would need validation on genuine local agronomic data before farmers act on its recommendations, transparent communication of the model's uncertainty (the class-probability output supports this), and safeguards so that automated "apply fungicide" recommendations do not encourage over-treatment — the recommendation layer intentionally frames Medium-risk outputs as "inspect" rather than "treat" for this reason.

Accessibility: the rule-based Decision Tree output was prioritised specifically so recommendations can be explained in plain language to users without a technical/statistical background.

8. Conclusion

This project designed, implemented and evaluated a complete pipeline — from data generation and cleaning through concept learning, decision trees, Bayesian classification, instance-based learning (KNN, LWR), a backpropagation-trained neural network and a genetic-algorithm-based feature selector — for crop disease-risk and yield-risk prediction. All required learning paradigms from CO6–CO8 were implemented from first principles

and cross-validated against reference library implementations. The Genetic Algorithm and the Decision Tree independently converged on the same two dominant risk drivers (humidity and rainfall), and the final integrated system combines a compact, interpretable classifier with a locally weighted yield regressor to produce actionable, farmer-readable recommendations. Requirements were achieved: working implementations of all ten questions, quantitative validation on held-out data, and a working end-to-end demonstration (Question 10). The principal limitation is that the underlying dataset is simulated rather than field-collected, so absolute performance numbers should be treated as a proof of concept pending validation on real agronomic data (see Reflection, below, for further limitations, uncertainty and bias–variance discussion).

9. Student Reflection

What I learned beyond the classroom: implementing Candidate-Elimination, Bayesian MLE, KNN, LWR and backpropagation entirely from first principles — rather than only calling library functions — made the underlying mathematics concrete in a way that using scikit-learn alone would not have. Cross-checking each manual implementation against a reference library implementation (e.g. my Bayesian classifier's accuracy matching GaussianNB exactly) was a genuinely useful way to build confidence that the derivations were correct, and is a habit I intend to keep for future ML work.

Model limitations and uncertainty: no model exceeded roughly 68% accuracy on the three-class disease-risk task, and the confusion matrix (Figure 5) shows most errors occur between the adjacent Medium and High classes rather than between Low and High — meaning the models are directionally reliable but imprecise near class boundaries, which is exactly where a real early-warning system needs to be most trustworthy. The Bayesian model's explicit class probabilities (rather than a single hard label) are one practical way to communicate this uncertainty to an end user, and are used in the Question 10 demonstration for that reason.

Bias–variance behaviour: this was directly observed in two experiments — the KNN validation curve (Figure 6), where $K=1$ is a low-bias/high-variance model that memorises training data (train accuracy 1.00) but generalises poorly (test accuracy 0.540), while $K=11$ trades a little bias for much lower variance and the best test accuracy (0.647); and the LWR bandwidth study (Figure 7), where a very small τ under-fits (high bias from over-localised, noisy local fits) and a very large τ approaches a single global linear model, with the best generalisation at an intermediate $\tau=1.0$ — a textbook illustration of the bias–variance trade-off in a locally weighted, non-parametric setting.

SDG 2 and SDG 13 reflection: working through this assignment clarified how directly a modest, interpretable ML pipeline (not necessarily the most complex model) can support food-security and climate-adaptation goals in practice — the Decision-Tree and GA feature-selection results here show that two easily-measured variables (humidity, rainfall) already carry most of the predictive signal, which is encouraging for low-cost, low-sensor deployments in resource-limited farming contexts.

If given more time/resources, I would: (1) replace the simulated dataset with a real, field-collected or open agronomic dataset to validate whether the same feature importances and error patterns hold; (2) add case-based reasoning (explicitly required by CO6 alongside KNN/LWR but not separately implemented here beyond KNN's instance-based mechanism) as a complementary explain-by-example layer; (3) explore ensemble methods (e.g. a small random forest) to see whether they close the accuracy gap with the MLP while retaining more interpretability than a single deep network; and (4) run a proper k-fold cross-validation rather than a single train/test split, to obtain confidence intervals on all reported metrics.

10. References

- [1] Mitchell, T. M. (1997). Machine Learning. McGraw-Hill. (Candidate-Elimination / Version Space algorithm; concept learning formulation.)
- [2] Pedregosa, F. et al. (2011). Scikit-learn: Machine Learning in Python. Journal of Machine Learning Research, 12, 2825–2830. (Used only to cross-validate the from-scratch Decision Tree and Bayesian implementations.)
- [3] Harris, C. R. et al. (2020). Array programming with NumPy. Nature, 585, 357–362.
- [4] The pandas development team. (2023). pandas-dev/pandas: Pandas (Documentation). <https://pandas.pydata.org/>
- [5] Hunter, J. D. (2007). Matplotlib: A 2D Graphics Environment. Computing in Science & Engineering, 9(3), 90–95.
- [6] United Nations. Sustainable Development Goals — SDG 2 (Zero Hunger) and SDG 13 (Climate Action). <https://sdgs.un.org/goals>
- [7] AI-assisted tools: Claude (Anthropic) was used as a coding and drafting assistant to help implement and document the analysis in this report; all code was executed and all figures/results verified locally before inclusion.